



DOCUMENTACIÓN TÉCNICA

Sistema de Inventario — Bodega #1

Arquitectura, modelo de datos y guía de mantenimiento

Documento dirigido al Departamento de TI

Elaborado por: J. Peña & S. Vines

FAINCA Group

Agosto de 2026

1. Resumen del sistema

El Sistema de Inventario FAINCA es una aplicación web Java EE (Jakarta) que gestiona el stock de la Bodega #1. Está construida sobre el patrón MVC clásico de Servlets + JSP, con acceso a datos mediante JDBC sobre MySQL, sin frameworks externos (no usa Spring, Hibernate ni similares).

Esta decisión fue deliberada: reduce la curva de aprendizaje para su mantenimiento futuro, elimina dependencias pesadas y mantiene el despliegue en un único archivo WAR autocontenido.

1.1 Ficha técnica

Elemento	Detalle
Lenguaje	Java 21
Tipo de proyecto	Aplicación web (WAR) gestionada con Maven
API web	Jakarta Servlet 5.0 (jakarta.servlet.*, no javax.servlet.*)
Vistas	JSP + JavaScript nativo (sin framework de frontend)
Base de datos	MySQL 8 — esquema fainca_inventario
Servidor de desarrollo	Jetty 11.0.24 (plugin Maven), puerto 3000
Contraseñas	BCrypt (at.favre.lib:bcrypt), factor de coste 10
Reportes	Apache POI 5.3.0 (Excel) y OpenPDF 1.3.35 (PDF)
Intercambio de datos	JSON mediante Gson 2.11.0

1.2 Volumen de código

Capa	Archivos	Detalle
Servlets	21	Controladores HTTP, todos heredan de BaseServlet
DAO	7	Acceso a datos; ProductoDAO y MovimientoDAO concentran la lógica
Objetos (modelo)	4	Producto, Movimiento, Usuario, Marca
Filtros	2	AuthFilter (seguridad) y CacheFilter (caché)
Reportes	2	ReporteExcel y ReportePdf
Utilidades	1	ImagenUtil (compresión de fotos)
Páginas JSP	13	Incluye sidebar.jsp reutilizado por todas
JavaScript propio	11	Un archivo por pantalla, más app.js compartido

2. Arquitectura

La aplicación se organiza en cinco capas. Cada petición del navegador atraviesa los filtros de seguridad, llega a un Servlet que valida los datos, y éste delega el acceso a la base de datos en la capa DAO. Ninguna capa superior consulta la base directamente.

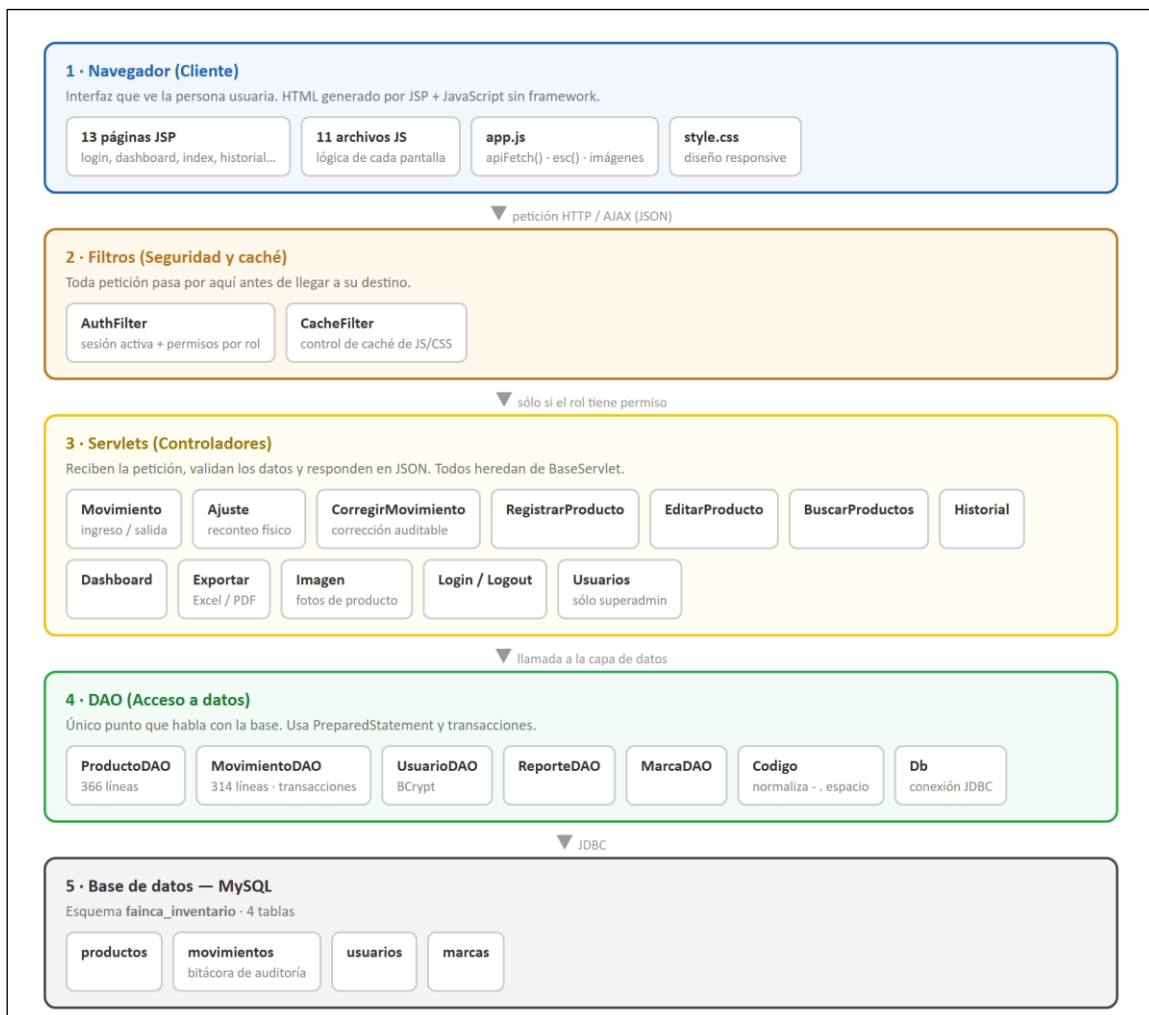


Figura 1 — Arquitectura en capas del sistema.

2.1 Estructura de paquetes

```
src/main/java/  
  Servlets/    Controladores HTTP (21 clases)  
  Dao/         Acceso a datos + Db, Codigo (7 clases)  
  Objetos/     Modelo de dominio (4 clases)  
  Filtros/     AuthFilter, CacheFilter  
  Reportes/    ReporteExcel, ReportePdf  
  Util/        ImagenUtil  
src/main/webapp/  
  *.jsp        13 vistas  
  js/          Lógica de cliente (11 archivos)  
  css/         style.css  
  images/      Logo institucional  
  vendor/      Librerías de terceros (FontAwesome, SweetAlert2)  
db/  
  schema.sql   Esquema inicial  
  migracion-*.sql Cambios posteriores al esquema
```

2.2 Convención de los Servlets

Todos los Servlets heredan de BaseServlet, que centraliza la lectura y escritura de JSON, el manejo de errores y la obtención del usuario en sesión. Esto evita repetir el mismo código en 21 clases:

```
public abstract class BaseServlet extends HttpServlet {
    protected JsonObject leerJson(HttpServletRequest request)
    protected void responderJson(HttpServletResponse response, Object cuerpo)
    protected void responderError(HttpServletResponse response, int codigo,
String mensaje)
    protected Usuario usuarioActual(HttpServletRequest request)
    protected String texto(JsonObject json, String campo)
    protected int entero(JsonObject json, String campo, int porDefecto)
}
```

3. Modelo de datos

El esquema consta de cuatro tablas. La tabla movimientos es el eje del sistema: funciona como bitácora de auditoría y registra absolutamente todo cambio de stock.

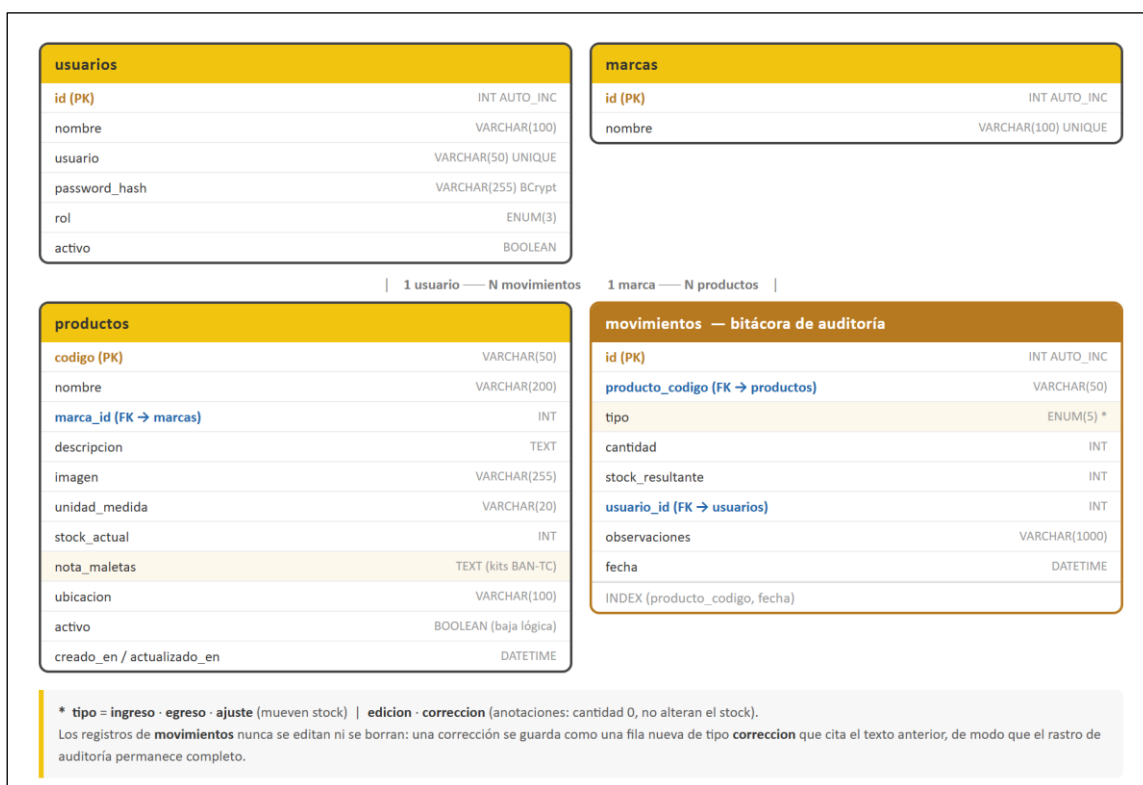


Figura 2 — Modelo entidad-relación del esquema fainca_inventario.

3.1 La tabla movimientos como bitácora

El campo stock_actual de la tabla productos nunca se modifica de forma directa desde ningún punto del código. Todo cambio pasa por MovimientoDAO, que actualiza el stock y escribe la fila correspondiente en movimientos dentro de la misma transacción. De este modo, el stock siempre es reconstruible a partir del historial y no puede existir un cambio sin su rastro.

Los cinco valores del campo tipo se dividen en dos grupos:

Tipo	Mueve stock	Descripción
ingreso	Sí (+)	Entrada de mercadería
egreso	Sí (–)	Salida o despacho
ajuste	Sí (±)	Reconteo físico; fija el stock en la cantidad real contada
edicion	No	Cambio en los datos del producto (marca, foto, ubicación, estado)
correccion	No	Aclaración sobre la observación de un movimiento anterior

Importante para consultas: Cualquier métrica que cuente «movimientos reales» debe excluir ambos tipos de anotación. Ver ReporteDAO.resumen(), que usa la condición tipo NOT IN ('edicion','correccion').

3.2 Bajas lógicas

Ni los productos ni los usuarios se eliminan físicamente de la base. Ambas tablas tienen una columna activo (BOOLEAN): al «eliminar», ésta pasa a 0. El registro deja de aparecer en los listados, pero conserva íntegro su historial y sus claves foráneas, evitando registros huérfanos en movimientos.

3.3 Migraciones

Los cambios de esquema posteriores a la instalación inicial se documentan como archivos SQL independientes en la carpeta db/, para poder aplicarlos de forma controlada en cada equipo:

Archivo	Cambio que aplica
schema.sql	Esquema inicial completo
usuarios-iniciales.sql	Carga de las cuentas de acceso iniciales
migracion-superadmin.sql	Añade el rol superadmin al ENUM de usuarios
migracion-correccion-observacion.sql	Añade el tipo correccion al ENUM de movimientos
migracion-quitar-stock-minimo.sql	Elimina la columna stock_mínimo (función descartada)

Al desplegar en un equipo nuevo: Ejecutar schema.sql y luego las migraciones en el orden listado. Si el código se actualiza sin aplicar la migración correspondiente, las operaciones afectadas fallarán.

4. Seguridad

4.1 Autenticación

Las contraseñas se almacenan como hash BCrypt con factor de coste 10; nunca en texto plano ni de forma reversible. La verificación ocurre en `UsuarioDAO.autenticar()`. Una sesión válida guarda el objeto `Usuario` en la `HttpSession`, y la sesión viaja al navegador en la cookie `JSESSIONID` gestionada por el contenedor.

4.2 Control de acceso por rol

`AuthFilter` intercepta todas las rutas (`@WebFilter(\`

```
// Rutas públicas (sin sesión iniciada)
PUBLICAS = { "/login.jsp", "/LoginServlet" }

// Rutas permitidas al rol ventas (además de las públicas)
VENTAS = { "/", "/index.jsp", "/BuscarProductosServlet", "/ProductosServlet",
           "/cambiarPassword.jsp", "/CambiarPasswordServlet", "/LogoutServlet",
           "/ImagenServlet", "/reportes.jsp", "/ExportarServlet" }

// Gestión de usuarios: sólo superadmin (ni siquiera un admin normal)
SOLO_SUPERADMIN = { "/usuarios.jsp", "/UsuariosServlet",
                    "/EditarUsuarioServlet", "/EliminarUsuarioServlet" }
```

Ante una petición no autorizada, el filtro distingue el tipo de cliente: si es una llamada AJAX responde con un código HTTP (401 si la sesión expiró, 403 si falta permiso) y un JSON de error; si es una navegación normal redirige al login o al índice. El lado del cliente (`app.js`, función `apiFetch`) detecta el 401 y devuelve al usuario a la pantalla de ingreso.

4.3 Filtrado de datos según el rol

El rol ventas no sólo tiene menos pantallas: además recibe menos campos. `BuscarProductosServlet.comoMapa()` omite deliberadamente los datos internos de bodega para ese rol, de modo que no viajan al navegador aunque se inspeccione la respuesta de red.

4.4 Prevención de inyección SQL y XSS

- SQL: todas las consultas usan `PreparedStatement` con parámetros vinculados. No se concatenan valores de usuario dentro de las sentencias.
- XSS: todo dato proveniente de la base se escapa con la función `esc()` de `app.js` antes de insertarse en el HTML (descripciones, observaciones, nombres de usuario, etc.).
- Servido de imágenes: `ImagenServlet` valida el nombre del archivo y rechaza cualquier ruta que contenga `«..»`, `«/»` o `«\»`, impidiendo el acceso a archivos fuera de la carpeta de imágenes.

5. Decisiones de diseño relevantes

Esta sección documenta las decisiones cuya razón no resulta evidente al leer el código, y que conviene conocer antes de modificarlo.

5.1 Normalización de códigos de producto

Los catálogos de los distintos proveedores escriben el mismo código de formas diferentes: con guion, con espacio o con puntos (WERMA usa códigos como 241.340.50). Para el sistema todos representan el mismo producto.

La clase Dao.Codigo centraliza esta regla en dos métodos gemelos: uno normaliza el valor buscado del lado de Java y el otro genera el fragmento SQL equivalente para comparar contra lo almacenado.

```
static String sqlNormalizado(String columna) {
    return "REPLACE(REPLACE(REPLACE(" + columna + ", '-', ''), ' ', ''), '.',
    '')";
}

static String normalizar(String codigo) {
    return codigo == null ? null
        : codigo.replace("-", "").replace(" ", "").replace(".", "");
}
```

Consecuencia práctica: Esta normalización se aplica en la búsqueda, en la detección de códigos duplicados al registrar y en la localización del producto al registrar un movimiento. Si en el futuro se debe ignorar otro carácter, basta con modificar esta única clase.

5.2 Integridad de fechas

No todos los movimientos admiten la misma libertad para editar su fecha, y la restricción se aplica en el servidor, no sólo en el formulario:

Operación	Regla de fecha	Dónde se aplica
Ingreso / Salida	Editable hasta 2 días atrás; nunca futura	MovimientoServlet.DIAS_MAXIMO_ATRAS
Ajuste	Siempre la fecha y hora actuales	AjusteServlet pasa null al DAO
Registro de producto	Siempre la fecha y hora actuales	RegistrarProductoServlet pasa null al DAO

El fundamento es que un recuento físico o el alta de un producto describen algo que ocurre en el momento; permitir retrofecharlos restaría fiabilidad a toda la bitácora. En cambio, un ingreso o una salida sí pueden registrarse con un par de días de retraso respecto al hecho físico.

Detalle de implementación: MovimientoServlet acepta únicamente los tipos ingreso y egreso. El tipo ajuste fue excluido a propósito de ese endpoint: de lo contrario se podría registrar un ajuste con fecha arbitraria, evitando la regla más estricta de AjusteServlet.

5.3 Correcciones auditables

Cuando una observación ya guardada contiene un error (por ejemplo, un número de guía equivocado), el sistema no permite editarla ni borrarla. MovimientoDAO.corregirObservacion() inserta una fila nueva de tipo corrección que cita el texto anterior y el corregido, junto con el usuario y la fecha de la corrección.

El tipo corrección existe como valor propio del ENUM en lugar de identificarse por el contenido del texto: un texto se puede imitar, un tipo no. Esto además permite filtrar todas las correcciones

desde la propia interfaz y las marca con un triángulo de advertencia, de modo que resulten visibles durante una auditoría.

5.4 Estrategia de caché de recursos estáticos

CacheFilter marca el JavaScript y el CSS propios como no-cache, lo que no significa «no almacenar» sino «preguntar siempre si cambió»: el navegador conserva el archivo y revalida con If-Modified-Since, recibiendo un 304 de pocos bytes cuando no hubo cambios. Las librerías de terceros bajo /vendor/, que nunca cambian, se cachean por un año.

Adicionalmente, las referencias a los archivos propios en los JSP llevan un sufijo de versión (por ejemplo style.css?v=24) que se incrementa en cada despliegue. Es la garantía de que ningún equipo de la red quede ejecutando una versión antigua tras una actualización.

5.5 Gestión de las fotografías

Las imágenes no se guardan en la base de datos ni dentro del directorio de la aplicación, sino en una carpeta externa configurable (app.imagenes.carpeta en db.properties, por defecto C:/fainca-inventario/imagenes). Así sobreviven a un redespliegue o recompilación del WAR.

ImagenUtil redimensiona cada imagen a un máximo de 800 px de lado y la comprime a JPEG con calidad 0,8 antes de guardarla. Una foto de varios megabytes queda típicamente en 80–150 KB, lo que mantiene ágil la carga de las páginas dentro de la red local.

6. Interfaz del sistema

A continuación se muestran dos pantallas representativas del sistema en funcionamiento, capturadas del entorno real.

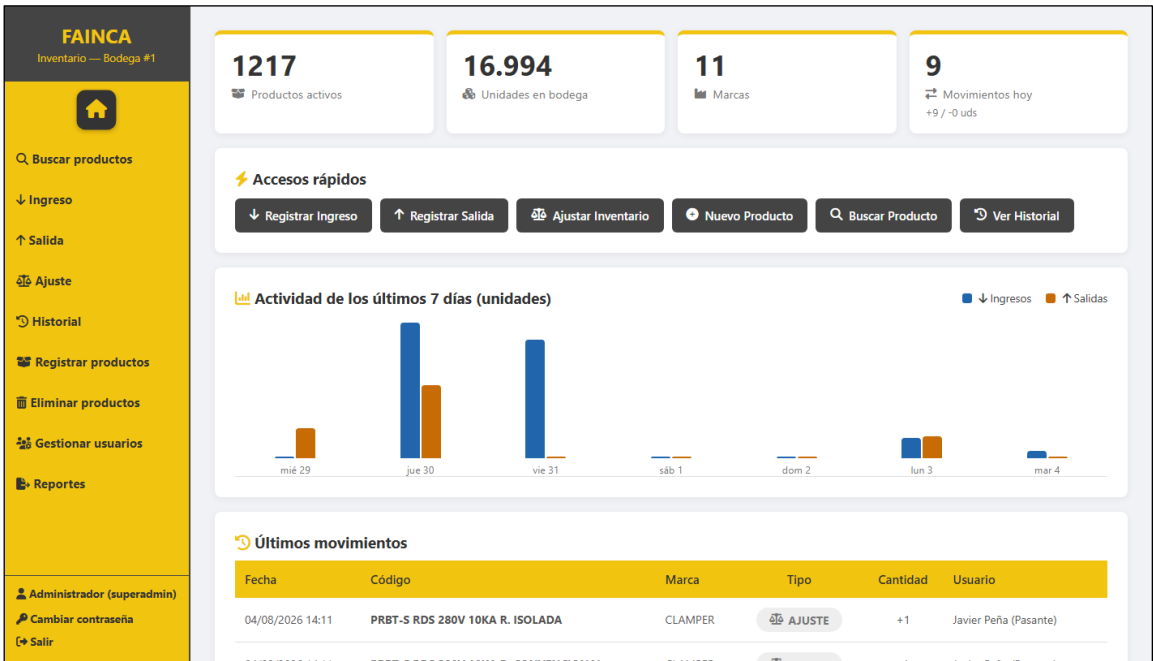


Figura 3 — Panel de control: indicadores, accesos rápidos, actividad semanal y últimos movimientos.

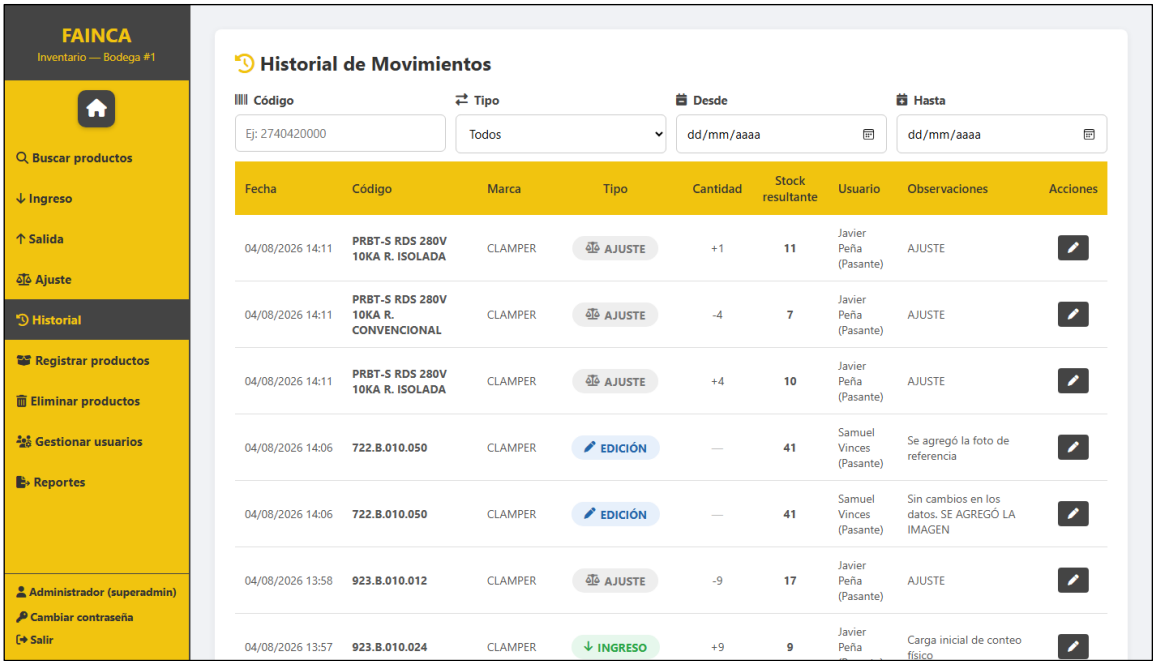


Figura 4 — Historial de movimientos: bitácora con filtros y botón de corrección auditable por fila.

7. Instalación y despliegue

7.1 Requisitos

- JDK 21 o superior.
- Apache Maven 3.9 o superior.
- MySQL 8 en ejecución, accesible desde el equipo servidor.
- Puerto 3000 disponible (configurable en el pom.xml mediante la propiedad «puerto»).

7.2 Configuración de la conexión

Los datos de conexión y la carpeta de imágenes se definen en `src/main/resources/db.properties`. Este archivo se mantiene fuera del código Java para no incluir credenciales en las clases compiladas:

```
db.url=jdbc:mysql://localhost:3306/fainca_inventario?useSSL=false&\
    allowPublicKeyRetrieval=true&serverTimezone=America/Guayaquil
db.usuario=root
db.password=*****
app.imagenes.carpeta=C:/fainca-inventario/imagenes
```

Recomendación de seguridad: Para el despliegue definitivo conviene crear un usuario de MySQL exclusivo para la aplicación, con permisos limitados al esquema `fainca_inventario`, en lugar de utilizar la cuenta `root`.

7.3 Puesta en marcha

El proyecto incluye el archivo `iniciar-servidor.cmd`, que sitúa la ruta de trabajo y arranca el servidor mostrando el registro en pantalla y guardándolo en `servidor.log`. Equivale a ejecutar:

```
cd <carpeta del proyecto, la que contiene pom.xml>
mvn jetty:run
```

El sistema está disponible cuando en el registro aparece la línea «Started ServerConnector ... 0.0.0.0:3000». A partir de ese momento se accede desde el propio equipo en `http://localhost:3000/` y desde el resto de la red en `http://[IP-DEL-SERVIDOR]:3000/`.

Advertencia operativa: La ventana del servidor no debe cerrarse: al hacerlo se detiene la aplicación y ningún equipo de la red podrá acceder. Para detenerlo de forma intencionada, se usa `Ctrl+C`.

7.4 Generación del paquete de distribución

```
mvn clean package -> target/fainca.war
```

El archivo WAR resultante puede desplegarse en cualquier contenedor compatible con Jakarta Servlet 5 (Tomcat 10 o superior, Jetty 11 o superior). Debe tenerse en cuenta que Tomcat 9 y anteriores no son compatibles, ya que utilizan el espacio de nombres `javax.servlet`.

8. Guía de mantenimiento

8.1 Puntos de entrada más frecuentes

Para modificar...	Archivos implicados
Una regla de negocio sobre movimientos	Dao/MovimientoDAO.java
Los permisos de un rol	Filtros/AuthFilter.java + sidebar.jsp
El comportamiento de la búsqueda de códigos	Dao/Codigo.java (única clase)
La apariencia general	webapp/css/style.css
Una pantalla concreta	webapp/<pantalla>.jsp + webapp/js/<pantalla>.js
El contenido de los reportes	Reportes/ReporteExcel.java, Reportes/ReportePdf.java
El esquema de la base de datos	Nueva migración en db/ + actualizar schema.sql

8.2 Procedimiento ante un cambio

- Detener el servidor antes de editar archivos JSP, JS o CSS: mientras se ejecuta, mantiene esos archivos bloqueados.
- Si el cambio afecta a JavaScript o CSS, incrementar el sufijo de versión (?v=N) en todos los JSP que los referencian.
- Si el cambio afecta al esquema, crear una migración SQL nueva en db/ y actualizar schema.sql para que ambos reflejen el mismo estado.
- Reiniciar el servidor y verificar que la compilación finaliza sin errores antes de dar por concluido el cambio.

8.3 Consideraciones para escalar

El sistema está dimensionado para el uso actual: una bodega, alrededor de 1.200 productos activos y una decena de personas usuarias en red local. Si en el futuro creciera el alcance, conviene revisar los siguientes puntos:

- Conexiones a la base: actualmente Db.getConnection() abre una conexión por operación. Con una concurrencia alta convendría incorporar un pool (HikariCP).
- Consulta del historial: está limitada a los 300 registros más recientes. Con un volumen mayor, procede implementar paginación.
- Sesiones: se mantienen en memoria del servidor. Un despliegue con varias instancias requeriría sesiones compartidas.
- Copias de seguridad: conviene programar un respaldo periódico del esquema fainca_inventario y de la carpeta de imágenes, que hoy no está automatizado.